INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

AP1S1 – Algoritmos e Programação Funções

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

Funções

- Modularizar é quebrar um programa em vários “programas” menores
- Em geral, problemas complexos exigem algoritmos complexos
  - Mas sempre pode ser possível dividir um problema grande em problemas menores → cada parte menor tem um algoritmo mais simples, mais fácil de ser elaborado
- Esse trecho menor é chamado de sub-rotina
  - No caso da linguagem Python, as sub-rotinas são chamadas de funções
- O uso de sub-rotinas permite tornar o código modular, onde estas podem ser executadas (chamadas) em diferentes partes do programa e quantas vezes quanto for necessário

Dividir para conquistar

- Exemplo:

```
print("Início")
print("Ola, bem-vindo!!!")
print("Esse é o meu programa!")

print("Programa em execução...")

print("Espero que tenha gostado!")
print("Obrigado por usar o meu programa")
print("Fim!")
```

Apresenta o programa

Se despede

Dividir para conquistar

- Com o uso de funções, podemos dar nome a trechos de código
- Após nomear trechos de códigos, podemos chamá-los no programa
- O exemplo anterior ficaria:

```
print("Início")
apresentar()
print("Programa em execução...")
despedir()
print("Fim!")
```

Devemos criar os códigos "apresentar()" e "despedir()"

Criando funções sem argumentos e sem retorno

- Primeiro define-se o nome da função, criando seu cabeçalho
  - Utiliza-se palavra reservada "**`def`**"
- Após isso, define-se o que a função deve realizar, criando o corpo da função

```
def apresentar():
    print("Ola, bem-vindo!!!")
    print("Esse é o meu programa!")
```

Nome da função (cabeçalho)

Recuo é obrigatório

Tarefas a realizar (corpo da função)

Criando funções sem argumentos e sem retorno

- Exemplo completo

```
def apresentar():
    print("Ola, bem-vindo!!!")
    print("Esse é o meu programa!")

def despedir():
    print("Espero que tenha gostado!")
    print("Obrigado por usar o meu programa")
```

Funções

```
print("Início")
apresentar()
print("Programa em execução...")
despedir()
print("Fim!")
```

Programa principal

Funções e programas

- Um programa completo é dividido em um programa principal e diversas funções (a quantidade que for necessária)
- O programa principal é aquele em que inicia a execução do programa e pode chamar as demais funções
- Durante a execução do programa, quando encontra-se a invocação (ou chamada) de uma função, a execução do programa principal é interrompida. A seguir, executa-se o corpo da função invocada. Ao terminar a função, o programa principal volta a ser executado no ponto em que foi interrompido.



Funções

- Para criar uma função em Python:

```
def nome_da_função ( parâmetros ):
    '''
    Comentários sobre a função.
    Embora opcionais são fortemente recomendados. Os
comentários devem descrever o papel dos parâmetros e
o que a função faz.
    '''
    # corpo da função
    |
    | bloco de comandos
    |
```

Vantagens do uso de funções

- Divide o programa em partes, o que facilita o gerenciamento e entendimento
- Evita repetição de código
- Pode deixar o código menor

Parâmetros em funções

- Forma de enviar valores para serem trabalhados pelas funções
- Os parâmetros são variáveis
  - Devem ter seu valor informado quando se chama a função
- Uma função pode receber vários parâmetros, dos mais diversos tipos

Funções que recebem parâmetros

```
def nome_da_funcao(<parâmetros>) {
   <comandos>
}
```

- nomeDaFuncao: identificador da função

- <parâmetros>: nomes dos parâmetros

INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

Funções que recebem parâmetros

- Exemplo: Função para somar dois números

```
def soma(a, b):
    r = a + b
    print(r)
```

Função

```
print("Bem-vindo ao programa")
soma(2, 3)
```

Programa principal

Retorno de funções

- As funções retornam um valor para quem chamou
- Entender as funções como “caixas-pretas”: você fornece alguns valores, e ela retorna um resultado de alguma operação sobre esses valores
- A função deve indicar o retorno com o comando return (valor a ser retornado)
- O comando return retorna o valor da função e volta para quem a chamou

Retorno de funções



Retorno de funções

- Exemplo de criação da função:

```
def soma(a, b):
    r = a + b
    return r
```

- Exemplo de uso da função:

```
resultado = soma(2, 3)
print(“Resultado da soma:”, resultado)
```

Esqueleto de um programa em Python

- Para escrever programas em Python utilizando funções pode-se utilizar o seguinte modelo de esqueleto: a primeira função sempre será chamada de main() e corresponderá a função principal do programa

- A função main() será a primeira função a começar a ser executada para resolver o problema

- As demais funções auxiliares devem ser definidas, em qualquer ordem, após a definição da main() e, por fim, para executar o programa, a função main precisa ser chamada

Esqueleto de um programa em Python

```
# função  principal
def main():
    '''
      Responsável pela chamada das subrotinas
    '''
    # corpo da função main



# Declaração das funções
def f( parâmetros_de_f ):
    '''
    docstring da função f
    '''
    # corpo da função f
    |
    | bloco de comandos
    |
```

Esqueleto de um programa em Python

```
def g( parâmetros_de_g ):
    '''
    docstring da função g
    '''
    # corpo da função g
    |
    | bloco de comandos
    |


[...]


# início da execução do programa
#-----------------------------------------------
main() # chamada da função main
```

Esqueleto de um programa em Python

- Exemplo: Operações com 2 números inteiros

```
def leitura():
  nro=int(input("Digite um numero inteiro: "))
  return(nro)

def soma(n1,n2):
  return n1+n2

def subtraia(n1,n2):
  return n1-n2

def multiplique(n1,n2):
  return n1*n2

def divida(n1,n2):
  return n1/n2
```

Esqueleto de um programa em Python

- Exemplo:

```
def main():
    op=input("Escolha uma operação (+, -, *, /): ")
    nro1=leitura()
    nro2=leitura()
    if op == '+':
       print('O resultado é: ',soma(nro1,nro2))
    elif op == '-':
       print('O resultado é: ',subtraia(nro1,nro2))
    elif op == '*':
       print('O resultado é: ',multiplique(nro1,nro2))
    elif op == '/':
       print('O resultado é: ',divida(nro1,nro2))
    else:
       print('Operação Inválida')
```

Esqueleto de um programa em Python

- Exemplo:

```
# início da execução do programa
#-----------------------------------------------
main() # chamada da função main
```

Funções Modificadoras

- Uma função é **modificadora** quando recebe uma lista como parâmetro e altera os valores dessa mesma lista dentro da função

- Como a lista de objetos é compartilhada por **duas referências**, se a função modifica os elementos da lista parâmetros, a função que fez a chamada também enxerga as alterações, já que as alterações estão ocorrendo no mesmo endereço de memória

Funções Modificadoras

- Exemplo:

```
def double_stuff(lista):
for posicao in range(len(lista)):
        lista[posicao] = 2 * lista[posicao]

things = [2, 5, 9]
print(things)
double_stuff(things)
print(things)
```

Funções Puras

- Uma função **pura** (*pure function*) não produz efeito colateral. Sua comunicação com a função que a chamou é somente através do parâmetros, que não são modificados e um valor é retornado. Exemplo:

```
def double_stuff(lista):

    nova_lista = []
    for valor in lista:
        novo_elemento = 2 * valor
        nova_lista.append(novo_elemento)
    return nova_lista
```

Funções Puras

- Uma função é pura quando recebe uma lista como parâmetro e NÃO altera os valores dessa mesma lista dentro da função

- Exemplos:

```
things = [2, 5, 9]
things_emdobro = dobra_lista(things)
print(things)
print(things_emdobro)
```

Funções que produzem listas

- A versão pura da função double_stuff acima faz uso de um padrão (*pattern*) que você deve guardar na sua caixa de ferramentas
- Sempre que você necessitar escrever uma função que cria e retorna uma lista, o padrão usualmente é:

```
inicialize uma variável para a lista nova com a lista vazia
itere
    crie um novo elemento
    append esse novo elemento a lista nova
retorne a lista nova
```

Funções que produzem listas

- Exemplo:

```
#Função: entrada=n e saída=lista_pares
def pares_ate(n):
    print("Lista Pares")
    lista_pares = []
    for i in range(2,n):
        if i % 2 == 0:
            lista_pares.append(i)
    return lista_pares

meus_pares = pares_ate(50)
print(meus_pares)
```

List comprehension

- *List comprehension* é uma maneira compacta de criar listas. A forma geral para obter esse efeito é:

```
[<expressão> for <item> in <sequência> if <condição>]
```

- Exemplo:

```
minha_lista = [1, 2, 3, 4, 5]

sua_lista = [item ** 2 for item in minha_lista]
print(minha_lista)
print(sua_lista)
```

List comprehension

- Exemplo:

```
def par(num):
  if num % 2 == 0:
     return True
  else:
     return False

def pares_ate(n):
    lista_pares = [num for num in range(2,n) if par(num)]
    return lista_pares

meus_pares=pares_ate(20)
print("Meus pares:", meus_pares)
```

Escopo de Variáveis

- O escopo de uma variável indica sua visibilidade no código, ou seja, onde a variável é acessível

- Temos dois escopos para variáveis em Python:
  - **global** e **local**

Funções e variáveis

- Programas podem ter variáveis que podem ser utilizadas em qualquer parte do código: **Variáveis Globais**
- As variáveis globais podem ser usadas por qualquer função
- Variáveis globais são declarada FORA das funções
- Programas podem ter variáveis proprietárias de uma função: **Variáveis Locais**
- As variáveis locais podem ser usadas e acessadas apenas dentro da função onde foram criadas
- Essa classificação de variáveis (globais ou locais) também é chamada de **ESCOPO** das variáveis.

Variável global e local

- **Variáveis globais:**
  - São criadas fora das funções
  - Podem ser acessadas por todas as funções presentes no programa onde estão definidas
- **Variáveis locais:**
  - São criadas dentro de uma função
  - Existem apenas dentro da função onde foi declarada
  - As variáveis locais são inicializadas a cada nova chamada à função
- **Obs**: em caso de conflito, ou seja, a variável local e a global terem nomes idênticos, vale a variável local

Exemplo de variável local e global

```
VAR_GLOBAL="Variavel definida fora da função"
def imprimir_frase():
    VAR_LOCAL="Variavel definida na função imprimir_frase()"
    print("Variável global: ", VAR_GLOBAL)
    print("Variável local: ", VAR_LOCAL)
print("Executando a função escreve_texto:")

###########PROGRAMA PRINCIPAL#################
imprimir_frase()
print("Tentando acessar a variável global no programa principal:")
print("Variável global: ", VAR_GLOBAL)
print("Tentando acessar a variável local no programa principal:")
print("Variável local: ", VAR_LOCAL)
```

Exemplo de variável local e global

- Teste o exemplo:

```
a=1
b=2

def soma():
    print(a)
    print(b)
    c=a+b
    print(c)
    return(c)

d=soma()
print("A soma é ",d)
print("A soma é ",c)
```

Exercícios

Para todos os exercícios a seguir as leituras e impressões de resultados devem ser realizadas no programa principal:

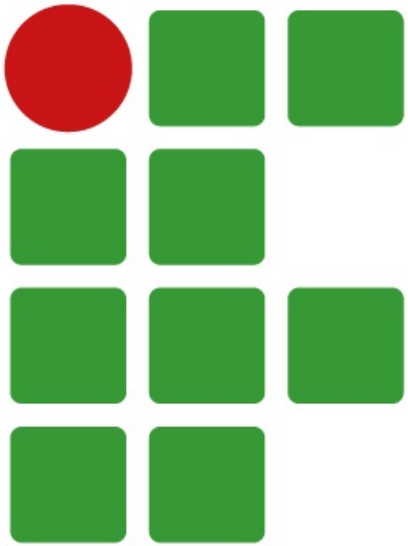
1. Faça uma função InteiroPositivo(n) que verifica se uma string fornecida como parâmetro pode ser convertida para um número inteiro positivo. A função deverá retornar True em caso afirmativo e False, caso contrário. O programa principal deverá fazer a conversão para inteiro, caso o resultado da função seja True, ou imprimir a mensagem adequada, caso o resultado seja False.
2. Faça uma função Inteiro(n) que verifica se uma string fornecida como parâmetro pode ser convertida para um número inteiro positivo ou negativo. A função deverá retornar True em caso afirmativo e False, caso contrário. O programa principal deverá fazer a conversão para inteiro, caso o resultado da função seja True, ou imprimir a mensagem adequada, caso o resultado seja False.
3. Faça uma função Real(n) que verifica se uma string fornecida como parâmetro pode ser convertida para um número real. A função deverá retornar True em caso afirmativo e False, caso contrário. O programa principal deverá fazer a conversão para inteiro, caso o resultado da função seja True, ou imprimir a mensagem adequada, caso o resultado seja False.

Exercícios

4. Crie uma função que receba como parâmetro um número inteiro. A função deve retornar um número inteiro, conforme a seguir:
   a) Retornar 1 se o número recebido é positivo
   b) Retornar -1 se o número recebido é negativo
   c) Retornar 0 se o número recebido é zero
5. Faça uma função que receba quatro valores reais (faça a consistência usando a função definida no exercício 3), referentes as notas que um aluno obteve nos bimestres. A função deve retornar a média final desse aluno.
6. Faça uma função que calcule o fatorial de um número inteiro positivo fornecido como parâmetro. Use a função definida no exercício 1) para verificar se o número é inteiro positivo. A função deverá retornar o fatorial do número, ou False (caso não seja um inteiro positivo). O resultado (ou a mensagem) deve ser impresso pelo programa principal.

Exercícios

7. Faça uma função que receba 2 listas de valores inteiros e retorne a lista UNIÃO.
8. Faça uma função que receba 2 listas de valores inteiros e retorne a lista INTERSECÇÃO
9. Faça uma função que receba 2 listas de valores inteiros e retorne a lista DIFERENÇA.
10. Faça uma função que receba 2 listas de valores inteiros, o modo de saída (U:união, I:intersecção, D:diferença) e retorne a lista resultante. Obs: a função deverá invocar as funções definida nos exercícios de 7 a 9, para calcular U, I ou D.
11. Faça uma função que receba 2 strings e retorne Verdadeiro se a segunda string é um anagrama da primeira, ou Falso caso contrário.



INSTITUTO FEDERAL

São Paulo

Câmpus São Carlos